Worksheet 02 — Meeting R

From the console to your first plot with the tree leaf data

Bill Perry

2026-06-26

In-class Activity 2: Data Visualization

Recap from Activity 1

- Collected pine needle samples from windward and leeward sides of trees
- Identified independent variable (wind exposure) and dependent variable (needle length)
- Measured needle lengths and recorded data
- Created basic visualizations
- Saved our data for further analysis

Today's Objectives

1. Implement data pipeline best practices
2. Apply controlled vocabulary and naming conventions
3. Create effective tables and visualizations
4. Customize plots for publication quality
5. Combine multiple plots into composite figures

How to use this worksheet

Work through each section at your own pace. Type or paste the code into your script file in Positron, run it line by line, and answer the questions in the spaces provided. Code blocks marked ▸ **Run this** are the ones you should execute. Blocks marked ✏️ **Your turn** ask you to write or modify something. You do **not** need to finish every section in one class period — the "Going further" section at the end is there if you want to explore.

Part 1 · Set up R and Positron

Download and install

You need two pieces of software — install them **in this order**:

1. **R** (the engine) — https://cran.r-project.org/
   - Choose the installer for your operating system (Windows, macOS, Linux)
   - Run it and accept all the defaults
   - *Can you find the file that was installed?*
2. **Positron** (the dashboard) — https://positron.posit.co/download.html
   - Install after R — Positron needs to find R already on your machine
   - Open Positron after installing and verify it finds R (look at the bottom status bar)

⚠️ **Watch out!** Install R *first*, then Positron. If Positron was already open, restart it after installing R.

Orient yourself in Positron
Positron has four panes you will use constantly. Find each one on your screen now:

Pane	Where	What it does
Editor	Top-left	Your scripts live here
Console / Terminal	Bottom	Where code runs and results appear
Variables / Session	Right	Every object you have stored
Plots	Right (tab)	Your figures appear here

✏️ **Your turn:** Click the **Console** tab. Type `1 + 1` and press Enter. What did R return?

```
Your answer:
```

Part 2 · Build your project folder

Why a project folder?
In R we keep everything for one project together in one folder. Every file path is then *relative* to that folder — no more `C:/Users/myname/Desktop/random_stuff/final_FINAL2.xlsx` nightmares.

Create the folder structure
On your computer, make a new folder called `tree_exercise` somewhere tidy (your Documents folder is fine). Inside it, create these four sub-folders:

```
tree_exercise/
├── data_raw/      <- the untouched Excel file goes here — never edit it
├── scripts/       <- your .R code goes here
├── figures/       <- plots you save go here
└── data_clean/    <- tidied files you create go here
```

> 💡 **Coming from Excel?** In Excel you tend to keep one giant workbook. In R we keep raw data read-only and write *new* clean files — so you can always trace your steps and never accidentally overwrite your original numbers.

Open the folder as your workspace in Positron
In Positron: **File → Open Folder…** and pick your `tree_exercise` folder.

You should see `tree_exercise` appear in the Explorer panel on the left.

✏️ **Your turn:** What is the full path to your `tree_exercise` folder on your computer?

```
Your answer:
```

Copy the data file
Copy the file `2026_06_25_tree_experiment_raw_data.xlsx` into `tree_exercise/data_raw/`.

> ⚠️ **Important:** Leave this copy alone — it is your raw data. We will never overwrite it.

Create your first script
In Positron: **File → New File → R File**. An untitled file opens in the Editor pane.

Save it immediately: **Ctrl/Cmd + S**, name it `01_tree_analysis.R`, and save it inside `tree_exercise/scripts/`.

> 💡 **Key idea:** Console = scratch paper. Script = the lab notebook you keep.

✏️ **Your turn:** What is your script named? Write it here:

```
Your answer:
```

Part 3 · How R works

R as a calculator
Type each line below into your **Console** (not the script yet) and press **Enter** after each one.

▸ **Run this in the Console:**

```
3 + 5
12 / 7
2 ^ 10
sqrt(144)
```

> ⚠️ **Watch out!** If you see a `+` at the *start* of a new console line instead of the `>` prompt, R is waiting for you to finish a command. Press **Esc** to cancel and start over.

✏️ **Your turn:** What does `2 ^ 10` return? What does the `^` operator do?

```
Your answer:
```

Storing values with `<-`
To keep a value, give it a **name** using the assignment operator `<-`. Shortcut: **Alt + –** (Windows) or **Option + –** (Mac) types `<-` for you.

▸ **Run this in the Console:**

```
x <- 7          # store 7 under the name x
x               # type the name to see it
x * 2           # use it in math
```

Look at the **Variables** pane on the right — `x` should appear there.

✏️ **Your turn:** Now store the number `42` under the name `my_number`, then multiply `my_number` by `x`. What is the result?

```
# Write your code here:
```

```
Your answer:
```

Naming things well

Good names make code readable months later. Rules:

- Be explicit but not too long: `leaf_mass` not `x2`
- Cannot start with a number — `x2` ✅, `2x` ❌
- R is **case-sensitive** — `weight_kg` ≠ `Weight_kg`
- Use **`lower_snake_case`** — words separated by underscores, all lowercase

✏️ **Your turn:** Which of these names are valid in R? Circle or write Y/N next to each.

```
leaf_mass       –
3rdleaf         –
Width_cm        –
my.leaf.data    –
side            –
```

Comments with `#`

Anything to the right of `#` is **ignored by R** — it is a note for humans. Shortcut to toggle a comment on/off: **Ctrl/Cmd + Shift + C**.

▸ **Run this in your Script:**

```
# weights of four leaves, in grams
leaf_mass <- c(4, 3, 5, 7)

mean(leaf_mass)   # average leaf mass
```

💡 **Key idea:** If a line confused you while writing it, write a comment explaining *why* you did it.

Functions and arguments

A **function** is a mini-program you call by name. You feed it **arguments** (inputs) inside `( )` and it returns a result.

▸ **Run this in the Console:**

```
sqrt(10)                        # one argument
round(3.14159)                  # default: 0 decimal places → 3
round(3.14159, 2)               # two decimal places → 3.14
round(x = 3.14159, digits = 2)  # same, with named arguments
```

To get help on any function:

```
?round          # open the help page
args(round)     # list what arguments it takes
```

✏️ **Your turn:** Use `round()` to round `pi` (R knows `pi` by name — just type it) to **4 decimal places**. Write your code and the result:

```
# Write your code here:
```

```
Your answer:
```

Vectors — a row of values

A **vector** is a series of values built with `c()` — the *combine* function. A spreadsheet column is really just a vector.

▸ **Run this in your Script:**

```
weight_g <- c(4, 3, 5, 7)          # numeric vector
side     <- c("sunny", "shady")    # character vector — needs quotes

length(weight_g)    # how many values?
class(weight_g)     # what type is it?
str(weight_g)       # structure overview
```

> ⚠️ **Watch out!** Quotes matter: `"sunny"` is text. Without quotes, R searches for an *object* named `sunny` and throws an error when it cannot find one.

✏️ **Your turn:** Make a character vector called `my_colors` with three color names of your choice. Then check its `length()` and `class()`.

```
# Write your code here:
```

Data types inside vectors

Every vector holds **one type**. The main ones you will use:

Type	Example	Notes
`numeric`	`4`, `3.5`	All numbers by default
`character`	`"sunny"`	Any text, always quoted
`logical`	`TRUE` / `FALSE`	Must be all-caps
`integer`	`2L`	Whole numbers only

▸ **Run this in the Console:**

```
class(c(1, 2, 3))
class(c("a", "b"))
class(c(TRUE, FALSE))
class(c(1L, 2L, 3L))
```

> ⚠️ **Watch out!** Mix types and R silently converts everything to the most flexible type — numbers become text, TRUE becomes 1. This can cause surprising bugs.

Part 4 · Packages and libraries

What is a package?

Base R is powerful but lean. **Packages** add new functions — like apps for your phone.

- **Install once** (downloads the package to your computer):

```
install.packages("tidyverse")
install.packages("readxl")
```

- **Load every session** (activates the package for today):

```
library(tidyverse)   # data wrangling + ggplot2 for plotting
library(readxl)      # reading Excel files
```

> ⚠️ **Watch out!** A very common first-day error is forgetting `library(tidyverse)`. If R says *"could not find function `read_excel`"*, you skipped the `library()` line.

▸ **Run this in your Script** (top of the script — packages always go at the top):

```
# ── Packages ─────────────────────────────────────────────────────────────
library(tidyverse)
library(readxl)
```

✏️ **Your turn:** What message does R print after `library(tidyverse)`? Write the first line here:

```
Your answer:
```

Part 5 · Load the tree data

Read the Excel file

▸ **Run this in your Script:**

```
# ── Load data ────────────────────────────────────────────────────────────
tree_df <- read_excel("data_raw/2026_06_25_tree_experiment_raw_data.xlsx")

tree_df    # print to the console
```

> 💡 **Coming from Excel?** `read_excel()` is the bridge from your spreadsheet into R. Your sheet's first row becomes the **column names**.

Notice the path `"data_raw/..."` is *relative* to your project folder — this works on anyone's computer, not just yours.

Meet the data frame

A **data frame** is R's word for a spreadsheet-style table. Each column is a vector of one type; the rows line up.

Our `tree_df` has five columns:

Column	Type	What it holds
`index`	numeric	leaf number (1–20)
`side`	character	`"sunny"` or `"shady"`
`weight_g`	numeric	leaf weight in grams
`width_cm`	numeric	leaf width in centimetres
`height_cm`	numeric	leaf height in centimetres

Pull a single column with `$`:

```
tree_df$weight_g     # just the weight column, as a vector
```

Always eyeball a new dataset

▸ **Run each of these in your Script:**

```
head(tree_df)       # first 6 rows
tail(tree_df)       # last 6 rows
dim(tree_df)        # (rows, columns)
names(tree_df)      # column names
str(tree_df)        # type of every column
summary(tree_df)    # min / mean / max per column
```

✏️ **Your turn:** Answer these questions from the output above.

```
How many rows does tree_df have?

How many columns?

What type does R report for `side`?

What is the mean weight_g?

What is the maximum height_cm?
```

> ⚠️ **Watch out!** If a number column shows as `character`, a stray letter or comma snuck into your spreadsheet. Check your raw data file.

The pipe `|>` also as `%>%`— read it as "then"

The pipe operator sends a result straight into the next function. Read it as the word **"then"**:

▸ **Run this in your Script:**

```
# Take tree_df, THEN group by side, THEN calculate mean weight
tree_df |>
  group_by(side) |>
```

```
  summarise(mean_weight = mean(weight_g),
            sd_weight   = sd(weight_g),
            n           = n())
```

💡 The older pipe `%>%` from the tidyverse does the same thing — you will see both in code you read online.

✏️ **Your turn:** Modify the `summarise()` call above to also calculate the mean `height_cm`. Add a line: `mean_height = mean(height_cm)`

What are the mean heights for sunny and shady leaves?

```
Sunny mean height_cm:
Shady mean height_cm:
```

Part 6 · Your first plot

ggplot builds pictures in layers
Every ggplot is built in layers, joined with `+`:

1. **`ggplot(data, aes(...))`** — which data frame, which columns map to x and y
2. **`geom_*()`** — the shape to draw (points, boxes, bars, ...)
3. **`labs()`** — axis labels and title

▸ **Run this in your Script — the simplest possible plot:**

```
ggplot(tree_df, aes(x = side, y = weight_g)) +
  geom_point()
```

⚠️ **Watch out!** The `+` must sit at the **end** of a line, never the start. A `+` at the start of a line causes an error.

Improve it one layer at a time
Since `side` is a category, a boxplot with raw points on top tells the story well.

▸ **Run this in your Script:**

```
ggplot(tree_df, aes(x = side, y = weight_g)) +
  geom_boxplot() +
  geom_jitter(width = 0.15, alpha = 0.6, color = "tomato") +
  labs(x     = "Side of tree",
       y     = "Leaf weight (g)",
       title = "Sunny vs. shady leaves")
```

✏️ **Your turn:** Make the same plot but for `height_cm` instead of `weight_g`. Copy the code above and change the relevant parts.

```
# Write your modified code here:
```

What pattern do you see — do sunny or shady leaves tend to be taller?

```
Your answer:
```

Save your plot
Store the plot as an object, then save it to your `figures/` folder.

▸ **Run this in your Script:**

```
# Save the weight plot
weight_plot <- ggplot(tree_df, aes(x = side, y = weight_g)) +
  geom_boxplot() +
  geom_jitter(width = 0.15, alpha = 0.6, color = "tomato") +
  labs(x     = "Side of tree",
       y     = "Leaf weight (g)",
       title = "Sunny vs. shady leaves")

ggsave("figures/leaf_weight.pdf",
       plot   = weight_plot,
       width  = 6,
       height = 6,
       units  = "in")
```

Check your `figures/` folder — the PDF should be there.

> ⚠️ **Watch out!** `ggsave()` wants the **filename first**, then `plot =`. Mixing that order is the classic first-save mistake.

Save a clean data file
Write a tidy CSV copy to `data_clean/` so future scripts can load it quickly.

▸ **Run this in your Script:**

```
write_csv(tree_df, "data_clean/tree_clean.csv")
```

Your project folder should now look like this:

```
tree_exercise/
├── data_raw/
│   └── 2026_06_25_tree_experiment_raw_data.xlsx   <- untouched original
├── scripts/
│   └── 01_tree_analysis.R                          <- your script
├── figures/
│   └── leaf_weight.pdf                             <- your saved plot
└── data_clean/
    └── tree_clean.csv                              <- the tidy copy
```

Part 7 · Review and checkpoint

At this point you can:

- ☐ Set up R and Positron and orient yourself in the four panes
- ☐ Build a project folder with `data_raw/`, `scripts/`, `figures/`, `data_clean/`
- ☐ Use R as a calculator and store values with `<-`
- ☐ Write a script with comments, save it, and re-run it
- ☐ Call functions with arguments and use `?` to get help
- ☐ Build numeric and character vectors with `c()`
- ☐ Install and load packages with `install.packages()` and `library()`
- ☐ Load an Excel file with `read_excel()` and inspect it with `str()` and `summary()`
- ☐ Use the pipe `|>` to group and summarise
- ☐ Build a `ggplot` boxplot with `geom_jitter()` and proper labels
- ☐ Save a plot to `figures/` with `ggsave()` and a CSV with `write_csv()`

✏️ **Your turn — before you move on:** Run the full script from top to bottom by pressing **Ctrl/Cmd + Shift + Enter** (runs the whole file). Does it complete without errors?

```
Did it run cleanly? Y / N
If not, what error did you see?
```

Part 8 · Going further — more plotting

> This section is optional — work through it if you finish early or want to explore. There are no wrong answers here; the goal is to see what ggplot can do.

Scatter plots — two numeric variables

▸ **Try this:**

```
ggplot(tree_df, aes(x = width_cm, y = height_cm, color = side)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(x     = "Leaf width (cm)",
       y     = "Leaf height (cm)",
       color = "Side of tree",
       title = "Leaf dimensions by side of tree")
```

✏️ **Your turn:** Does width predict height? Do the two groups (sunny/shady) cluster separately or overlap?

```
Your observation:
```

Add a trend line

▸ **Try this:**

```
ggplot(tree_df, aes(x = width_cm, y = height_cm, color = side)) +
  geom_point(size = 3, alpha = 0.7) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(x     = "Leaf width (cm)",
       y     = "Leaf height (cm)",
```

```
       color = "Side of tree",
       title = "Leaf dimensions with regression lines")
```

`method = "lm"` fits a linear model. `se = TRUE` draws the confidence ribbon.

✏️ **Your turn:** Do the slopes look similar or different for sunny vs. shady leaves? What might that mean biologically?

```
Your observation:
```

Violin plots

▸ **Try this:**

```
ggplot(tree_df, aes(x = side, y = weight_g, fill = side)) +
  geom_violin(alpha = 0.5) +
  geom_jitter(width = 0.1, size = 2) +
  labs(x     = "Side of tree",
       y     = "Leaf weight (g)",
       title = "Violin plot of leaf weight") +
  theme(legend.position = "none")   # legend is redundant here
```

✏️ **Your turn:** What does the width of the violin show you that a boxplot does not?

```
Your observation:
```

Facets — small multiples

Facets split one plot into panels, one per group.

▸ **Try this:**

```
ggplot(tree_df, aes(x = weight_g)) +
  geom_histogram(binwidth = 1, fill = "steelblue", color = "white") +
  facet_wrap(~ side, ncol = 1) +
  labs(x     = "Leaf weight (g)",
       y     = "Count",
       title = "Distribution of leaf weight by side")
```

✏️ **Your turn:** Do the histograms look roughly symmetric, or skewed? Are the shapes similar between sunny and shady?

```
Your observation:
```

Change the theme

ggplot comes with several built-in themes. Try swapping `theme_grey()` (the default) for others.

▸ **Try these one at a time by adding to any plot above:**

```
+ theme_bw()          # clean, white background
+ theme_minimal()     # very minimal, no border
+ theme_classic()     # classic axis lines, no grid
+ theme_dark()        # dark background
```

✏️ **Your turn:** Which theme do you prefer for this kind of data? Why?

```
Your answer:
```

Save two more plots

▸ **Try this:**

```
# Scatter plot saved as a PNG (good for documents, web)
scatter_plot <- ggplot(tree_df, aes(x = width_cm, y = height_cm, color = side)) +
  geom_point(size = 3, alpha = 0.7) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(x = "Leaf width (cm)", y = "Leaf height (cm)", color = "Side") +
  theme_bw()

ggsave("figures/leaf_dimensions_scatter.png",
       plot   = scatter_plot,
       width  = 7,
       height = 5,
       units  = "in",
       dpi    = 300)    # 300 dpi = publication quality
```

✏️ **Your turn:** What is the difference between saving as `.pdf` vs. `.png`? When might you prefer each?

```
Your answer:
```

What your finished project folder looks like

After working through this worksheet your `tree_exercise/` folder should contain:

```
tree_exercise/
├── data_raw/
│   └── 2026_06_25_tree_experiment_raw_data.xlsx   <- never touch this
├── scripts/
│   └── 01_tree_analysis.R                          <- your complete script
├── figures/
│   ├── leaf_weight.pdf
│   └── leaf_dimensions_scatter.png                 <- (Going further)
└── data_clean/
    └── tree_clean.csv
```

This is the project structure we will use **for every analysis** in this course:

Folder	Contents	Rule
`data_raw/`	Original files as received	**Read only — never overwrite**
`scripts/`	`.R` files	One script per analysis stage
`figures/`	Plots	Generated by scripts, never hand-edited
`data_clean/`	Processed data	Written by scripts, used by later scripts

Getting unstuck

When code breaks — and it will, that is normal — try these in order:

1. **Read the error message out loud.** R usually names the line and the problem.
2. **Check the usual suspects:** Did you load `library(tidyverse)`? Spelling? Missing `)` or `+` at the *start* of a line?
3. `?function_name` opens the built-in help page.
4. **Tidyverse cheat sheets** are invaluable — find them at https://posit.co/resources/cheatsheets/
5. **Bring the exact error** (copy-paste it) to class or office hours.

> 💡 **Key idea:** Every working data scientist googles error messages daily. Getting stuck is not failing — it is the job.

End of Worksheet 02. Next: Worksheet 03 will cover data cleaning, filtering with `filter()`, and your first t-test.